

Open in app ↗



Medium



Search



Write



Bancolombia Tech

Pruebas Continuas de Software: Una cultura estratégica para responder a la velocidad que requiere un banco con calidad y seguridad



Mauricio Puerta Ospina

Follow

10 min read · Dec 14, 2020



117





By: Pixabay FotoMix — Company

Continuous Testing: A strategic culture that a bank require to fast answers with quality and security.

Hace unos años el concepto de **Calidad de Software** era conocido como la práctica sistemática dentro de las compañías donde se realizaban pruebas de software manuales, que arrojaban un veredicto que se componía en: un documento con los escenarios, evidencias y un análisis de resultados de pruebas que, permitían determinar si el concepto de calidad de una aplicación era: **Favorable o No Favorable**.

En su momento, esta práctica era viable porque no teníamos presentes enfoques como:

- Automatización de Pruebas
- Métricas Automáticas
- DevSecOps
- RPA
- Monitorización
- Observabilidad
- Chaos Engineering

Aproximadamente hace tres años, en Bancolombia comenzamos a centrar la potencialización de los productos y el negocio con base a la tecnología, entendiendo que la transformación tecnológica nace desde adentro, lo cual trajo resultados muy positivos.

Con lo anterior, desde hace varios años comenzamos a implementar la Automatización de Pruebas, lo cual desplazaba la práctica manual a un segundo plano en la realización de pruebas de software. En Bancolombia, la **automatización permitió mejorar notablemente la velocidad de muchos desarrollos realizados**, respondiendo con la velocidad que necesita el negocio y exponiendo productos con grandes mejoras para que los usuarios y clientes tengan una mejor experiencia de servicio.

¿Cómo adoptar la Automatización de Pruebas en un banco?

Cuando hablamos de tecnología en un banco, pensamos en sistemas con interfaces a blanco y negro y letras verdes similares a la película de Matrix.

Esto está relacionado con las tecnologías legadas, sistemas complejos que acarrean burocracia, excesos de seguridad y limitaciones técnicas.

Sin embargo, la estabilidad y confianza que obtiene un banco a lo largo del tiempo es con base a la seguridad y la información, lo cual, hace cuidar mucho los sistemas que apalancan esta confianza del cliente en Bancolombia.

Por lo tanto, lo anterior nos permite pensar en, ¿cómo innovar en sistemas legados y complejos? La respuesta es simple: Modernizamos sin miedo al fracaso, apalancados en actores con grandes conocimientos en el negocio y en tecnología.

Un factor determinante que permite modernizar con confianza, es la adopción de pruebas automatizadas, donde lo correcto, debe continuar así, y cualquier cambio en la aplicación no debe afectar este comportamiento. Dicha garantía solo la puedo tener si realizo pruebas.

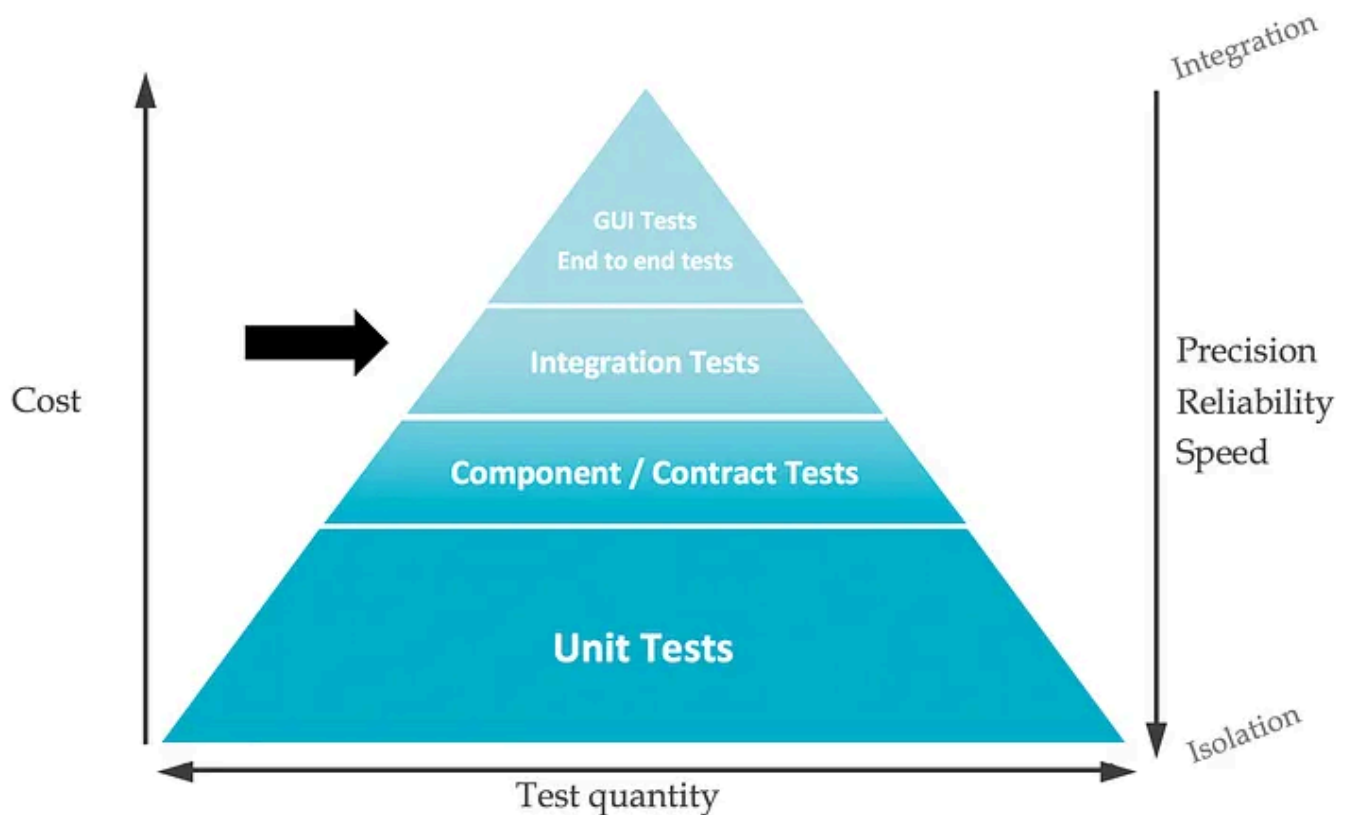
El diferencial está en cuánto podría costar volver a probar completamente una aplicación de software manualmente vs automáticamente. No solo el costo es económico, sino de tiempo, esfuerzo y falsos positivos que pueden desencadenar en riesgos potenciales de fallos en producción.

Por la anterior razón, en Bancolombia realizamos un esfuerzo enorme en potenciar el uso de la Automatización de Pruebas en todos los proyectos e iniciativas que se tuvieran al interior. A su vez, comenzamos a exigir a los aliados o terceros la entrega de componentes de software altamente garantizados con estándares de alta calidad definidos por el mismo Bancolombia.

¿Estándares en las Pruebas Automatizadas?

En Bancolombia somos fieles creyentes que para una compañía tan grande e importante se deben tener estándares que permitan medir el comportamiento de lo implementado. Así que, la inmersión de estrategias como lo son el **TDD**, **BDD** y **ATDD** se fueron añadiendo en la adopción del agilismo al interior de los equipos como motor de facilitación en entregas de valor al negocio.

Algo muy importante por mencionar, es que nuestra estrategia de calidad está basada en la pirámide de Cohn:



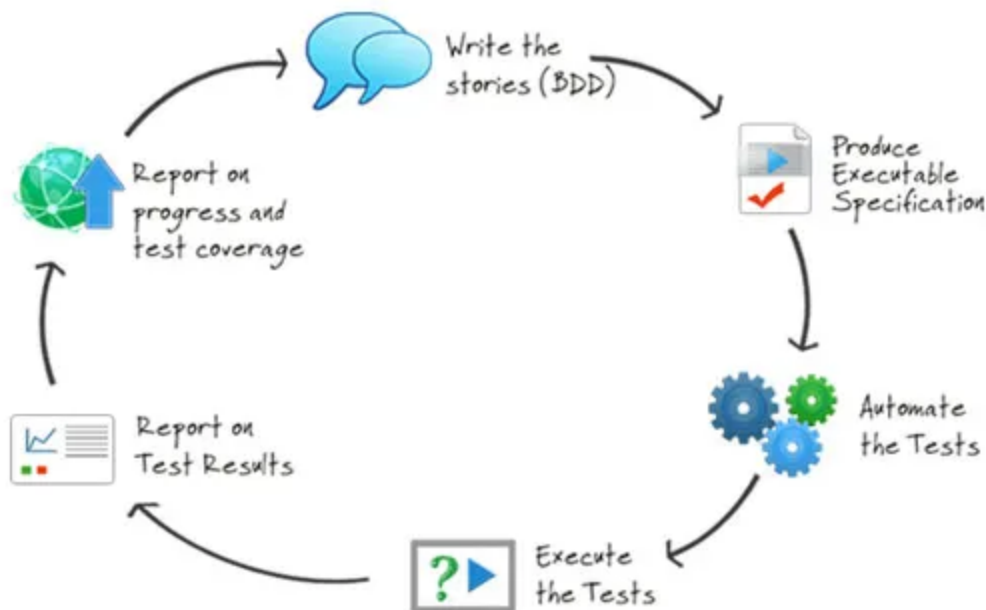
Pirámide de Pruebas — By: log.octo.com

A continuación, hablaremos de las diferentes capas de la pirámide de arriba hacia abajo:

- **E2E Tests (UI)**

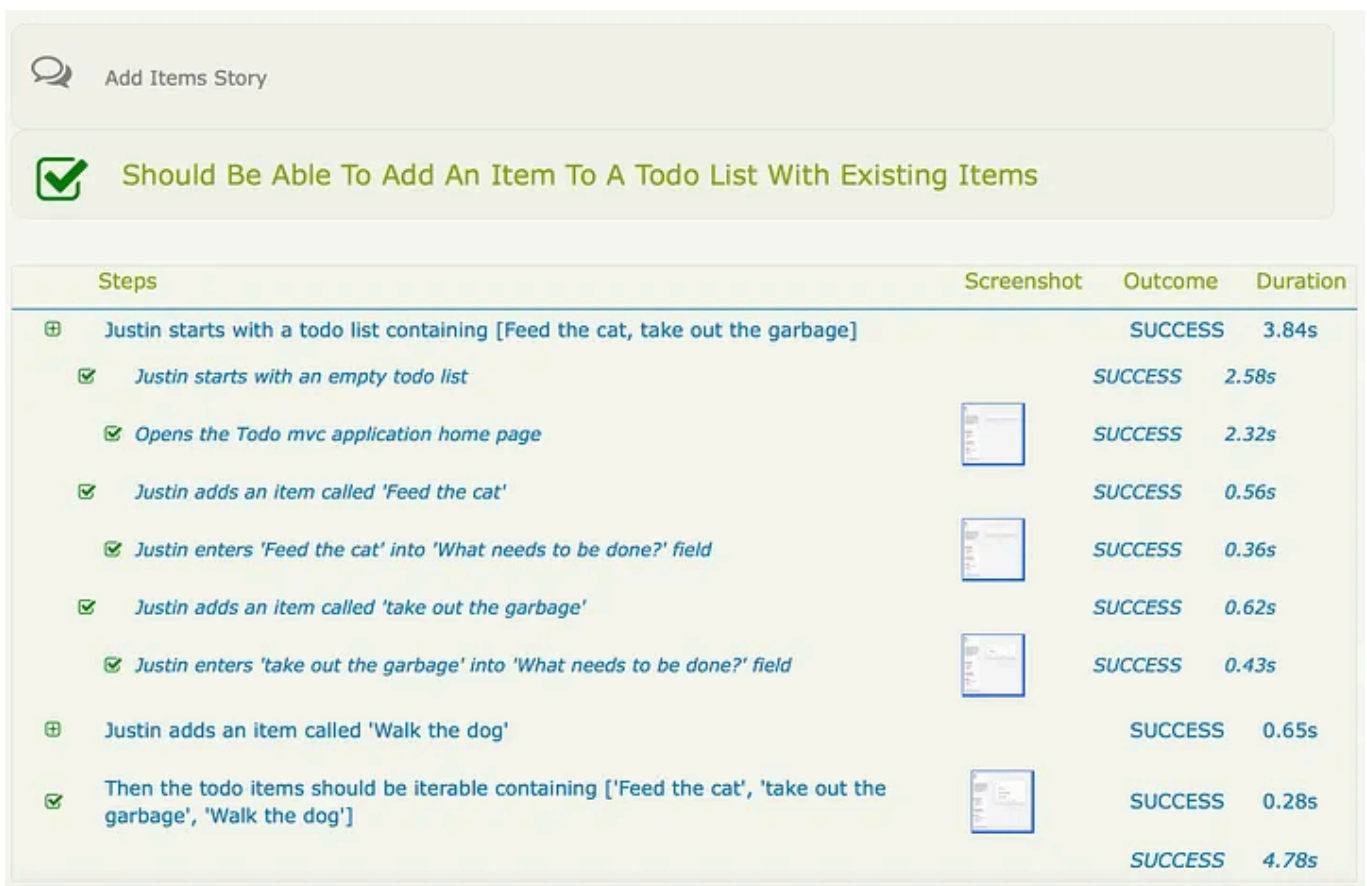
Estas pruebas por lo general son de carácter UI para las aplicaciones que tienen interfaz gráfica ya sea web, desktop ó mobile. También, tenemos pruebas de carácter E2E para componentes como lo pueden ser APIs, servicios, entre otros componentes que no exponen interfaz gráfica donde generalmente son llamadas Pruebas Subcutáneas.

Para esta capa de pruebas, adoptamos el BDD, que nos ayuda con la colaboración permanente entre actores principales del negocio y el equipo de tecnología, donde se definen los criterios de aceptación que deben conllevar los escenarios de pruebas. Son obligatorias en todo componente desarrollado y buscan abarcar todos los flujos críticos e importantes que se deben contemplar en cada cambio de la aplicación, para garantizar su funcionalidad transversal.



SerenityBDD By **John Ferguson Smart**

Para la realización de las pruebas E2E Automatizadas, tenemos una estrategia muy marcada con frameworks y herramientas con lo son: Java, Gradle y SerenityBDD, con la actuación estelar de Gherkin como DSL para la definición de los escenarios y cucumber para el entendimiento de los pasos a realizar en los escenarios de pruebas a nivel de código. Cabe aclarar, que la elección de SerenityBDD como framework de diseño y ejecución de las pruebas se debe a su adaptabilidad al patrón Screenplay y su generación de reportería que promueve el concepto de documentación viva.



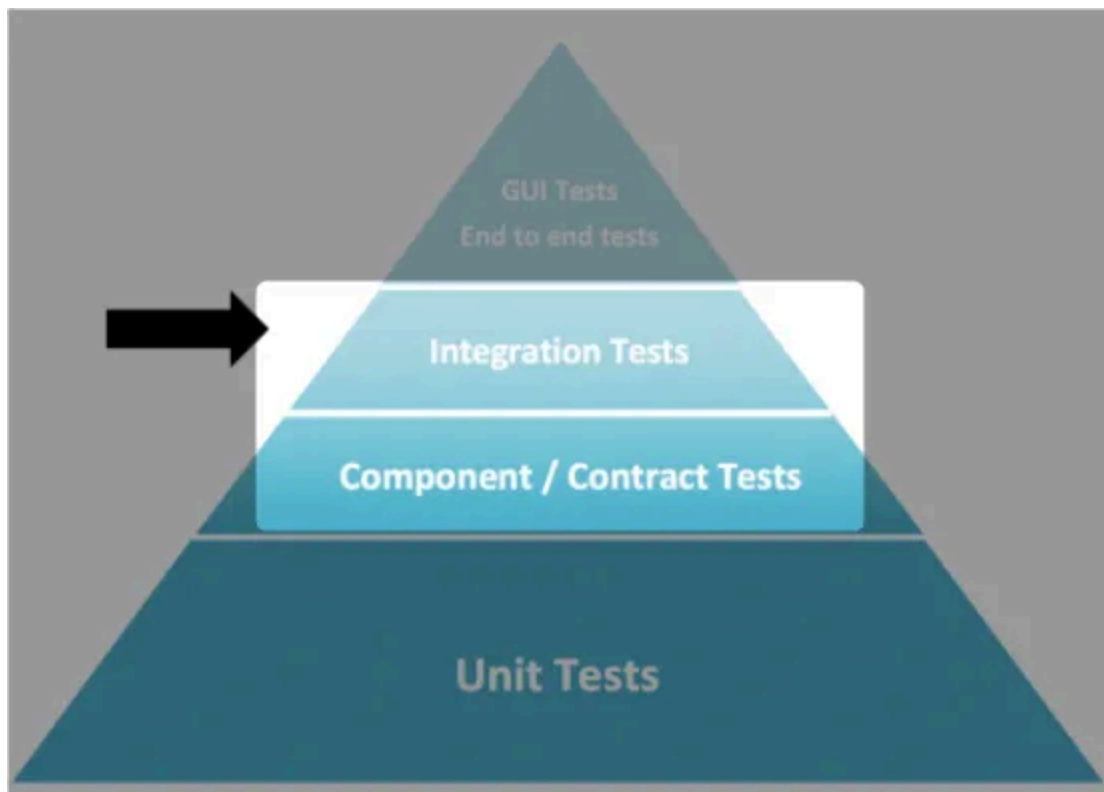
Add Items Story				
✓	Should Be Able To Add An Item To A Todo List With Existing Items			
Steps	Screenshot	Outcome	Duration	
⊕ Justin starts with a todo list containing [Feed the cat, take out the garbage]		SUCCESS	3.84s	
✓ Justin starts with an empty todo list		SUCCESS	2.58s	
✓ Opens the Todo mvc application home page		SUCCESS	2.32s	
✓ Justin adds an item called 'Feed the cat'		SUCCESS	0.56s	
✓ Justin enters 'Feed the cat' into 'What needs to be done?' field		SUCCESS	0.36s	
✓ Justin adds an item called 'take out the garbage'		SUCCESS	0.62s	
✓ Justin enters 'take out the garbage' into 'What needs to be done?' field		SUCCESS	0.43s	
⊕ Justin adds an item called 'Walk the dog'		SUCCESS	0.65s	
✓ Then the todo items should be iterable containing ['Feed the cat', 'take out the garbage', 'Walk the dog']		SUCCESS	0.28s	
		SUCCESS	4.78s	

Reporte de SerenityBDD by **John Ferguson Smart**

Como había mencionado anteriormente, hace más de 3 años, las pruebas eran manuales y se generaba un documento extenso donde documentaban las evidencias de manera manual.

Con la automatización de pruebas, el reporte de documentación viva reemplaza toda la documentación. Esto recalca nuestra ideología de brindarles a los actores principales de estas pruebas la opción de tener más tiempo para analizar, para automatizar y así generar mucho más valor, allí es donde somos más valiosos.

- **Integration Tests**



Piramide de Pruebas — By: log.octo.com

En esta capa de la pirámide, nuestras pruebas también son automáticas. En un principio, usábamos herramientas como Postman, Curl, Insomnia, entre otras, para probar de manera manual nuestros servicios Soap, Rest o WebServices generales. Esta práctica evolucionó conjuntamente con las antes mencionadas y se convirtió en Automatización para seguir generando valor al negocio y usuarios.

Actualmente, dentro de nuestro Stack Tecnológico contamos con herramientas y frameworks que nos permiten realizar estas pruebas de integración como lo son: Karate Framework, Soap UI y RestAssured embebido en SerenityRest. Con estas opciones podemos probar automáticamente servicios Rest, Soap y GraphQL.

Dentro de esta capa contamos con: **pruebas de contrato para APIs** con estrategias como Consumer Driven Contract, **pruebas de componentes para BDs y microservicios** usando TestContainers, Stubs, TestDoubles, y otras más. También, las pruebas integrales que nos permiten probar de manera activa como fueron desplegados los servicios y si cumplen con los criterios definidos, entre ellos: StatusCode, mensajería, excepciones, entre otros.

— **Contract Test:** Actualmente, usamos Spring Cloud Contract para realizar las pruebas de contrato, ya que la mayoría de nuestros componentes están basados en Java con Spring Framework. También, tenemos disponible la práctica para usar PACT como herramienta para estas pruebas, pero nuestro mayor esfuerzo está sobre SCC.

— **Component Test:** Usamos la librería opensource de TestContainers para simular y probar los componentes adjuntos en los microservicios. También, hacemos uso de funcionalidades propias de los frameworks de desarrollo con SpringBootTest, Junit, Mockito, Sinon, Jest, Chai, entre otros.

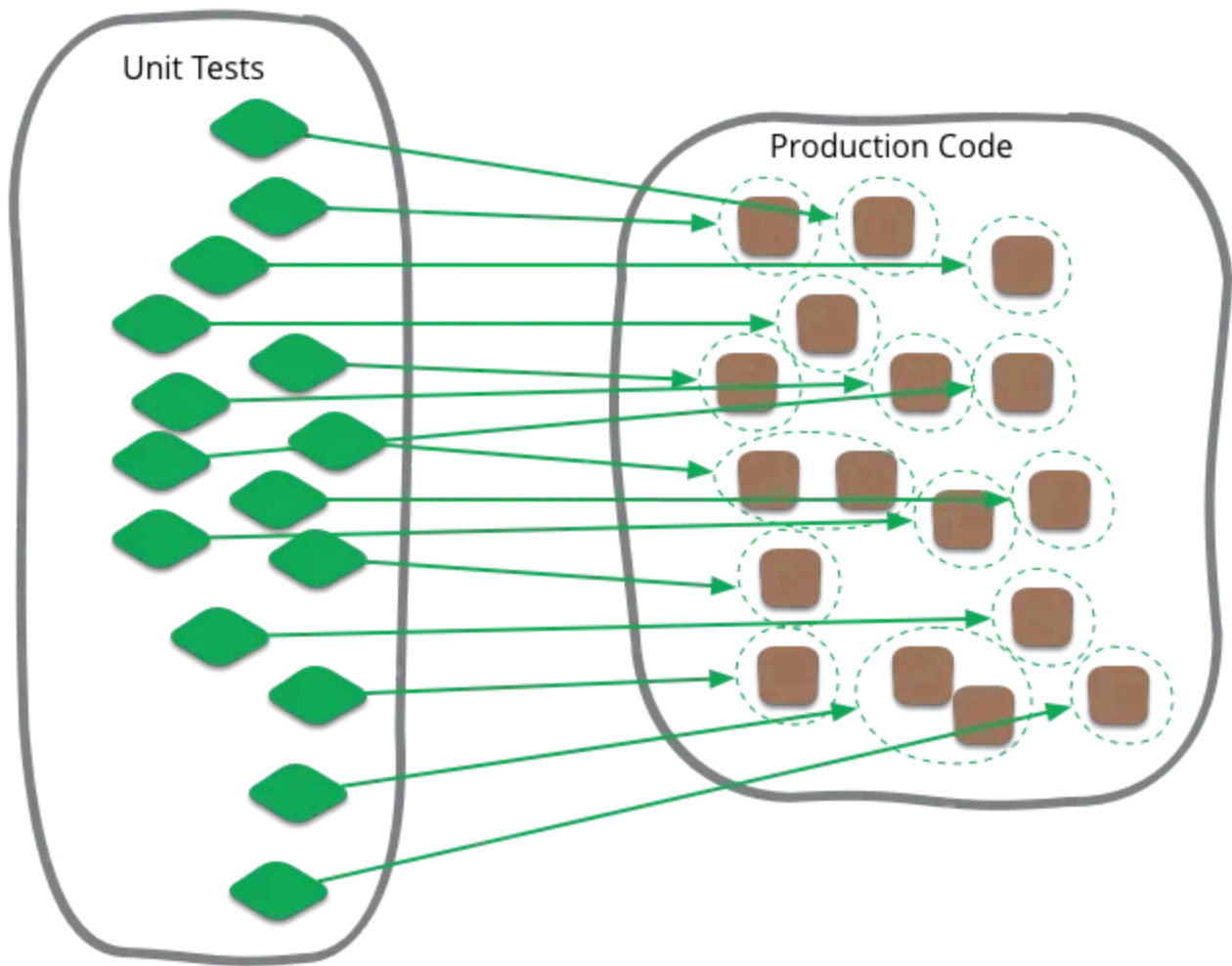
— **Integration Test:** Predomina el uso de SoapUI para estas pruebas, pero actualmente estamos promoviendo el uso de Karate Framework con librería opensource y que tiene amplia cobertura en funcionalidades de pruebas, con fácil adaptación a la documentación con BDD la cual pregonamos constantemente. También, tenemos componentes con pruebas en

RestAssured que realmente es un wrapper cuando la usamos con SerenityRest.

- **Unit Tests**

Actualmente, contamos con un stack tecnológico muy amplio para la realización de pruebas unitarias, todas automáticas, con el fin de abarcar la mayoría de lenguajes de programación con los que contamos, algunos por mencionar son: Java, Javascript, Python, .NET, Cobol, Elixir, Dart, entre otros. También, contamos con una diversidad de Frameworks que nos permiten promover el desarrollo de software con arquitecturas limpias, código testeable y buenas prácticas a nivel unitario.

Entre los frameworks que usamos para pruebas unitarias tenemos: Jest para aplicaciones en Angular y del lado del servidor con NodeJs, Junit para aplicaciones en Java, Pytest para pruebas en Python, MSTest para código en .NET y muchas otras herramientas que nos permiten testear a nivel unitario nuestros desarrollos para aplicaciones Web, Desktop y Mobile.



Unit Test solitario y sociable By **Martin Fowler**

Dentro de los enfoques con que abarcamos las pruebas unitarias, tenemos pruebas solitarias, y sociables.

Dentro de las solitarias, probamos todas las funcionalidades en nuestro código que no dependen de otras partes como lo pueden ser funciones, clases o procedimientos. Para estas particularidades probamos directamente el objeto de prueba y comparamos las diferentes salidas que esperamos obtener.

Ahora, dentro de las sociables, analizamos el código que requiere de otras fracciones de código que no necesariamente están alojadas en el mismo archivo. Aquí se toma la decisión si diseñar dobles de prueba para testear y

no tener dependencias o si realmente hacemos uso del código real para validar el resultado de la pruebas. Elegir esto depende del estado actual de las porciones de código externas y que tan bien probadas están. Por eso, el análisis es una práctica muy importante.

— TDD como buena práctica de desarrollo y testing

Recomendamos y promovemos el uso del TDD (Tests Driven Development) al interior de los equipos de TI, con el fin de mejorar la implementación y construcción de los mismos desarrollos y apalancar principios y patrones importantes desde la Arquitectura.

Para nadie es un secreto que, dejar las pruebas para la última fase de desarrollo de software, se puede convertir en cuellos de botella y en omisiones por premuras al salir con ello a producción. Por eso, la mejor manera de comenzar un desarrollo podría ser con las pruebas, debido a qué, si las pruebas fallan al principio del desarrollo, al no satisfacer los criterios de aceptación con las cuales fueron construidas y por ende, requiere solución, esto ayuda notablemente al desarrollador en su nivel de confianza para el despliegue, porque tiene tiempo de hacer los cambios necesarios para que su desarrollo sea lo más impecable posible.

Aparte de la recomendación que hacemos de aplicar TDD como algo cualitativo, ya que no podemos medir a alguien que use TDD o no, tenemos herramientas de análisis estático que nos permiten identificar el porcentaje de cobertura que tiene cada componente, y para ello establecemos unos Quality Gates que contienen reglas las cuales no se pueden omitir al momento de querer promover el código.

Algunas de las reglas que tenemos son: $\geq 70\%$ de cobertura para las pruebas unitarias y una **debt** menor a 2 días para el código. También, en esta fase del flujo aplicamos reglas de seguridad que nos ayudan apalancar la cultura DevSecOps que más adelante profundizaremos.

Cabe resaltar que no nos guiamos solamente por cumplir con un indicador, pues es bien sabido que los indicadores de coberturas están diseñados para cubrir escenarios de muy alto nivel, por lo cual, tratamos de dar conciencia a los equipos, que las pruebas deben ser exhaustivas y probar lo que verdaderamente debemos probar, tanto escenarios de éxito, como negativos y alternos.

- **Pruebas especializadas**

Unos de los aspectos más importantes en aplicaciones financieras son la seguridad y el *performance*, pues son cualidades que generan confianza y experiencia de usuario.



Las pruebas de performance se ejecutan en dos contextos:

- El primero son pruebas modulares, donde el desarrollador se encarga de construir el script de pruebas que le permita conocer al momento de despliegue de los cambios que los requisitos No Funcionales se estén cumpliendo acorde a lo definido en lo modular al componente objeto de pruebas.
- Y el segundo contexto son pruebas de performance E2E, que buscan analizar en caso de impacto, como se comporta el cambio que se quiere desplegar con respecto a toda la aplicación de principio a fin y en relación a parámetros No Funcionales como *Throughputs*, uso de recursos, y todos los criterios de rendimiento que arrojen como necesidad el análisis previo.

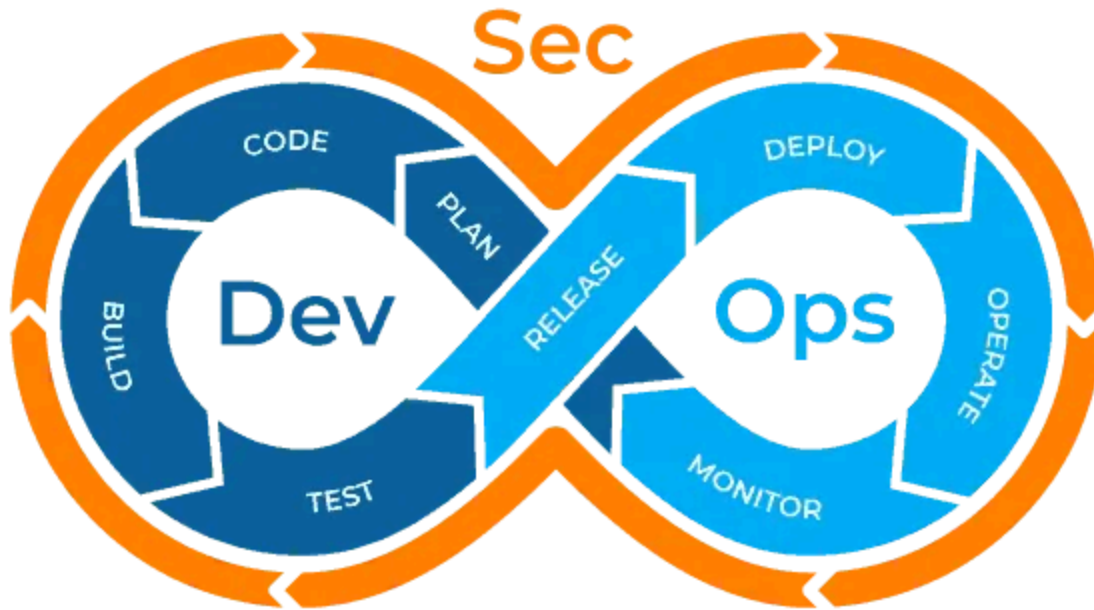
Para estas pruebas hacemos uso de herramientas como JMeter, SilkPerformer, Dynatrace para monitoreo, entre otras.

Sobre pruebas de seguridad hablaremos a continuación cuando se mencione todo lo relacionado a DevSecOps.

Cultura DevSecOps para promover el Continuous Testing

Todo lo mencionado en los anteriores apartados, no sería posible sin tener una cultura DevOps. Pues las herramientas, técnicas y frameworks que apalancan esta cultura, permiten la ejecución de las pruebas en toda acción que se requiera a nivel de despliegue y cambio de versiones.

El ciclo de pruebas continuas se evidencia desde que hacemos un commit y el sistema de integración continua autovalida los cambios, hasta que el cambio es desplegado a producción con tareas de *smoketest* que validan la aplicación y corroboran que su despliegue fue exitoso.



DevSecOps By: **Netec**

Desde hace 3 años se viene adoptando esta cultura al interior de Bancolombia, evolucionando hasta el punto de poder hablar de la aplicabilidad de prácticas de seguridad en todo el ciclo continuo de despliegue y pruebas.

Una de las prácticas de seguridad es Hacking Continuo, donde en cada despliegue en ambientes No Productivos, realizamos las pruebas de seguridad y garantizamos que las vulnerabilidades abiertas sean cerradas, o tengan su respectivo tratamiento en base a lo inspeccionado. Para la implementación usamos herramientas como OWASP ZAP en el pipeline de despliegue a nivel modular y también hacemos uso de aliados estratégicos

como Fluid Attacks que nos proveen pruebas de seguridad exhaustivas en un enfoque E2E.

Conclusión

Sin lugar a dudas, el panorama de hace 5 años, era completamente diferente al actual en una manera muy positiva. Aún, Bancolombia trabaja con sistemas legados, pero la orientación al mundo cloud es inevitable y ya contamos con una cantidad considerable de componentes migrados y sistemas completamente cloud, también híbridos entre On-Premise y Cloud.

Dicha orientación a la nube, no sería posible si las prácticas de pruebas al interior de los desarrollos no evolucionarán a nivel paralelo como la arquitectura, infraestructura y desarrollo como tal. Por eso, promovemos completamente la automatización de las pruebas y el cumplimiento estricto de este pilar.

Les dejo un artículo de como Bancolombia realiza Más de 10 despliegues al día en Producción en algunas aplicaciones, lo que refleja lo antes mencionado y promueve el concepto de probar rápido, fallar rápido, volver a probar y generar valor constante.

DevOps

+ Cloud

Software Engineering

+ Software Testing

+ Banking Technology

**Published in Bancolombia Tech**

824 followers · Last published Jun 12, 2026

Following